

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

**Отчет
Домашняя работа №4**

Выполнил:

Алексеев Андрей

Группа: К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках домашней работы требовалось спроектировать разбиение монолитного backend-приложения на микросервисы по принципу database-per-service, описать взаимодействие сервисов, выделить отдельные базы данных, подготовить межсервисные API-контракты и сформулировать конкретный план перехода от монолита к микросервисной архитектуре.

Исходная система

В качестве исходной системы рассматривается backend сервиса поиска работы, реализованный в lab1 на Spring Boot 3 и Kotlin. Монолит покрывает базовые сценарии job board: регистрацию и вход пользователей, роли APPLICANT, EMPLOYER, ADMIN и DICTIONARY_EDITOR, компании и профили работодателей, резюме соискателей, вакансии и назначения HR, отклики, избранное, историю просмотров, а также справочники индустрий, уровней опыта и навыков.

В монолите присутствуют следующие прикладные группы endpoint-ов:

- /auth
- /users
- /roles
- /companies
- /employer-profiles
- /industries
- /experience-levels
- /skills
- /resumes
- /vacancies
- /vacancy-assignments
- /applications
- /vacancies/favorites
- /vacancies/history

При проектировании новой архитектуры было важно не дробить приложение механически по контроллерам. Цель разбиения - выделить самостоятельные бизнес-возможности, сохранить понятные границы владения данными и при этом добавить Kafka там, где событийная модель действительно снижает связность сервисов.

Целевая микросервисная архитектура

В целевом варианте система разделяется на десять микросервисов. Такое количество достаточно для демонстрации микросервисной архитектуры и database-per-service, но не создает искусственные сервисы без самостоятельной ответственности.

Микросервис	Ответственность	Данные
api-gateway	Единая точка входа, маршрутизация, проверка JWT, CORS, rate limiting	Не владеет доменными данными
auth-service	Регистрация, вход, refresh token, выпуск JWT	Учетные данные и refresh tokens
user-service	Профили пользователей, роли, назначение ролей	Пользователи, роли, user-role
company-service	Компании и employer profiles	Компании и профили работодателей
vacancy-service	Создание, изменение, публикация вакансий и vacancy assignments	Вакансии, назначения HR, справочники вакансий
search-service	Публичный поиск вакансий, фильтрация, сортировка, пагинация	Поисковая read-модель опубликованных вакансий
resume-service	Резюме соискателей, образование, опыт, навыки резюме	Резюме и связанные таблицы
application-service	Отклики, статусы откликов, приглашения на собеседование	Отклики и история статусов
interaction-service	Избранные вакансии и история просмотров	Favorites и vacancy views
notification-service	In-app уведомления пользователям	Уведомления и статусы прочтения

Итоговая логическая схема взаимодействия выглядит следующим образом:

```
Client
-> api-gateway
   -> auth-service
   -> user-service
   -> company-service
   -> vacancy-service
   -> search-service
   -> resume-service
   -> application-service
   -> interaction-service
   -> notification-service
```

```
Kafka:
  user.events
  company.events
  vacancy.events
  resume.events
  application.events
  interaction.events
  notification.events
```

Разделение vacancy-service и search-service

Ключевое архитектурное решение - разделить сервис вакансий и сервис поиска. Vacancy-service остается источником правды по вакансиям: он отвечает за создание,

редактирование, удаление, публикацию вакансий, правила видимости и назначения работодателей. Search-service не владеет вакансиями как доменными сущностями, а хранит поисковую проекцию опубликованных вакансий.

Такое разделение соответствует подходу CQRS: запись и доменные инварианты находятся в vacancy-service, а чтение списка вакансий, фильтрация, сортировка и полнотекстовый поиск вынесены в search-service. Внешний URL GET /api/v1/vacancies можно сохранить прежним, но на уровне gateway этот маршрут будет направляться в search-service. Получение конкретной карточки вакансии по GET /api/v1/vacancies/{vacancyId} остается в vacancy-service.

Операция	Сервис-владелец	Причина
POST /vacancies	vacancy-service	Создание вакансии меняет доменную модель
GET /vacancies/{id}	vacancy-service	Нужна актуальная карточка и правила видимости
PATCH /vacancies/{id}	vacancy-service	Изменение вакансии и проверка прав работодателя
DELETE /vacancies/{id}	vacancy-service	Удаление или архивирование вакансии
GET /vacancies	search-service	Список вакансий, поиск, фильтры, сортировка и пагинация
GET /search/vacancies	search-service	Явная поисковая ручка для внутреннего или внешнего клиента

Search-service обновляет свою read-модель асинхронно. После публикации или изменения вакансии vacancy-service отправляет событие в Kafka, а search-service обрабатывает его и обновляет индекс. Поэтому между изменением вакансии и появлением изменений в поисковой выдаче допускается небольшая задержка. Это нормальное следствие eventual consistency.

Описание микросервисов и endpoint-ов

api-gateway

Gateway не содержит бизнес-логики и не имеет собственной доменной БД. Он принимает публичные запросы, проверяет JWT, маршрутизирует их в нужный сервис и скрывает внутреннюю топологию системы.

- Маршрутизация /api/v1/auth/** в auth-service
- Маршрутизация /api/v1/users/** и /api/v1/roles/** в user-service
- Маршрутизация write/read-by-id ручек вакансий в vacancy-service
- Маршрутизация GET /api/v1/vacancies в search-service

auth-service

Auth-service отвечает за регистрацию, вход, refresh token и выпуск JWT. Роли пользователя хранятся в user-service, поэтому при логине auth-service получает их синхронным внутренним запросом.

- POST /auth/register
- POST /auth/login
- POST /auth/refresh

user-service

User-service отвечает за профиль пользователя и роли. В него переносятся операции /users/me, /roles и назначение ролей пользователю.

- GET /users/me
- PATCH /users/me
- GET /roles
- POST /roles
- POST /users/{userId}/roles
- GET /internal/users/{userId}/auth-context

company-service

Company-service владеет компаниями и профилями работодателей. Он используется vacancy-service для проверки, что работодатель имеет право создать или изменить вакансию от имени компании.

- POST /companies
- GET /companies/{companyId}
- PATCH /companies/{companyId}
- POST /employer-profiles
- GET /employer-profiles/me
- GET /internal/employer-profiles/by-user/{userId}

vacancy-service

Vacancy-service владеет вакансиями и назначениями HR. Он проверяет бизнес-правила публикации, видимости и управления primary assignment.

- POST /vacancies
- GET /vacancies/{vacancyId}

- PATCH /vacancies/{vacancyId}
- DELETE /vacancies/{vacancyId}
- GET /vacancies/my
- GET /vacancies/{vacancyId}/assignments
- POST /vacancies/{vacancyId}/assignments
- PATCH /vacancy-assignments/{assignmentId}
- DELETE /vacancy-assignments/{assignmentId}
- GET /internal/vacancies/{vacancyId}/application-context

search-service

Search-service владеет поисковой проекцией вакансий. Он принимает события из Kafka и обновляет индекс опубликованных вакансий.

- GET /vacancies
- GET /search/vacancies
- POST /internal/search/reindex/vacancies/{vacancyId}

resume-service

Resume-service владеет резюме соискателей. Application-service обращается к нему для проверки, что резюме существует и принадлежит текущему пользователю.

- GET /resumes
- POST /resumes
- GET /resumes/{resumeId}
- PATCH /resumes/{resumeId}
- DELETE /resumes/{resumeId}
- GET /internal/resumes/{resumeId}/ownership

application-service

Application-service владеет откликами, их статусами и приглашениями на собеседование. Он не хранит вакансии и резюме целиком, а проверяет их через внутренние API vacancy-service и resume-service.

- POST /vacancies/{vacancyId}/applications
- GET /vacancies/{vacancyId}/applications
- GET /applications/my
- PATCH /applications/{applicationId}/status

- POST /applications/{applicationId}/interview-invitations

interaction-service

Interaction-service отвечает за избранные вакансии и историю просмотров. Просмотры также публикуются в Kafka для будущей аналитики и рекомендаций.

- GET /vacancies/favorites
- POST /vacancies/favorites/{vacancyId}
- DELETE /vacancies/favorites/{vacancyId}
- GET /vacancies/history
- POST /vacancies/history/{vacancyId}

notification-service

Notification-service создает in-app уведомления на основе событий из Kafka: новый отклик, просмотр отклика, изменение статуса, приглашение на собеседование, назначение HR на вакансию.

- GET /notifications
- PATCH /notifications/{notificationId}/read
- PATCH /notifications/read-all

Разделение базы данных

В целевой архитектуре каждый микросервис владеет своей базой данных или как минимум отдельной схемой в общем PostgreSQL-кластере на этапе локальной разработки. Прямые SQL-запросы к таблицам другого сервиса запрещены. Доступ к чужим данным осуществляется через внутренний REST API или через события Kafka.

База данных	Сервис	Основные таблицы
auth_db	auth-service	credentials, refresh_token, login_audit
user_db	user-service	user_account, role, user_role
company_db	company-service	company, employer_profile
vacancy_db	vacancy-service	vacancy, vacancy_assignment, vacancy_skill, industry, experience_level, employment_type, work_format
search_db / search_index	search-service	vacancy_search_document или индекс vacancies_index в OpenSearch
resume_db	resume-service	resume, resume_education, resume_work_experience, resume_skill
application_db	application-service	application, application_status_history, interview_invitation
interaction_db	interaction-service	favorite_vacancy, vacancy_view
notification_db	notification-service	notification, notification_delivery_attempt

Справочники не выделяются в отдельный микросервис. Они не образуют самостоятельный бизнес-процесс и используются как часть сценариев вакансий и резюме. Поэтому справочники, связанные с вакансиями, размещаются в vacancy-service. Это снижает количество сетевых вызовов и упрощает согласованность данных.

Кafka и событийное взаимодействие

Kafka используется для распространения доменных событий и обновления read-моделей. Синхронные запросы применяются только там, где сервису необходимо немедленно принять решение: проверить роль пользователя, принадлежность резюме, публикационный статус вакансии или права работодателя.

Topic	Producer	Consumer	События
user.events	user-service	auth-service, notification-service	UserProfileUpdated, RoleAssigned
company.events	company-service	search-service	CompanyCreated, CompanyUpdated
vacancy.events	vacancy-service	search-service, notification-service, application-service	VacancyCreated, VacancyUpdated, VacancyPublished, VacancyArchived, VacancyDeleted, VacancyAssignmentCreated
resume.events	resume-service	recommendation/read-model consumers в будущем	ResumeCreated, ResumeUpdated, ResumeDeleted
application.events	application-service	notification-service	ApplicationCreated, ApplicationViewed, ApplicationStatusChanged, InterviewInvitationCreated
interaction.events	interaction-service	analytics/recommendation consumers в будущем	VacancyViewed, VacancyAddedToFavorites, VacancyRemovedFromFavorites

Пример обработки публикации вакансии:

1. Employer отправляет запрос `PATCH /vacancies/{id}` со статусом `PUBLISHED`.
2. `Vacancy-service` проверяет права работодателя и сохраняет изменение в `vacancy_db`.
3. `Vacancy-service` публикует событие `VacancyPublished` в topic `vacancy.events`.
4. `Search-service` читает событие и создает/обновляет поисковый документ.
5. `Notification-service` может создать уведомления для подходящих кандидатов.
6. Пользователь видит вакансию через `GET /vacancies` после обновления поискового индекса.

Синхронное межсервисное взаимодействие

Несмотря на наличие Kafka, часть проверок должна выполняться синхронно, потому что без результата проверки невозможно корректно завершить пользовательский запрос.

Источник	Целевой сервис	Endpoint	Назначение
auth-service	user-service	GET /internal/users/{userId}/auth-context	Получение ролей и статуса пользователя при выпуске JWT
vacancy-service	company-service	GET /internal/employer-profiles/by-user/{userId}	Проверка employer profile и компании при создании вакансии
application-service	resume-service	GET /internal/resumes/{resumeId}/ownership	Проверка, что резюме принадлежит соискателю
application-service	vacancy-service	GET /internal/vacancies/{vacancyId}/application-context	Проверка, что вакансия опубликована и доступна для отклика
interaction-service	vacancy-service	GET /internal/vacancies/{vacancyId}/public-context	Проверка существования вакансии при добавлении в избранное или историю
notification-service	user-service	GET /internal/users/{userId}/notification-context	Получение минимального профиля получателя уведомления

OpenAPI-контракт межсервисных endpoint-ов

Для синхронного межсервисного взаимодействия подготовлен отдельный файл `openapi-microservices.yaml`. В нем описаны внутренние endpoint-ы, используемые сервисами для проверки прав, получения контекста пользователя, проверки резюме и вакансии, а также ручка принудительной переиндексации поисковой проекции.

Все внутренние endpoint-ы используют технический заголовок `X-Service-Token`. Он нужен, чтобы отделить межсервисные запросы от пользовательских публичных запросов и не открывать внутренние проверки напрямую наружу.

```
X-Service-Token: service-to-service-token
Authorization: Bearer <user-jwt> # передается только если нужно сохранить
пользовательский контекст
```

Примеры запросов и ответов

Проверка контекста пользователя

```
GET /internal/users/0b25f7c2-05e8-4a5f-b2d3-dc6e0a08e002/auth-context
X-Service-Token: service-to-service-token

200 OK
{
  "userId": "0b25f7c2-05e8-4a5f-b2d3-dc6e0a08e002",
  "email": "applicant@example.com",
  "roles": ["APPLICANT"],
  "active": true
}
```

```
}
```

Проверка резюме перед откликом

```
GET /internal/resumes/0f6dd3f1-5cb0-4e70-9f2d-
cfb5fc590a33/ownership?userId=0b25f7c2-05e8-4a5f-b2d3-dc6e0a08e002
X-Service-Token: service-to-service-token

200 OK
{
  "resumeId": "0f6dd3f1-5cb0-4e70-9f2d-cfb5fc590a33",
  "ownerUserId": "0b25f7c2-05e8-4a5f-b2d3-dc6e0a08e002",
  "ownedByUser": true,
  "status": "ACTIVE"
}
```

Проверка вакансии перед откликом

```
GET /internal/vacancies/71d50a83-475a-44cf-a9ab-048b3e9b80a4/application-context
X-Service-Token: service-to-service-token

200 OK
{
  "vacancyId": "71d50a83-475a-44cf-a9ab-048b3e9b80a4",
  "companyId": "9612ac62-191e-4c6f-ad61-19b4315e9342",
  "status": "PUBLISHED",
  "acceptsApplications": true,
  "assignedEmployerUserIds": ["d79d1a3b-7254-4dcc-aac4-3eb9418730ea"]
}
```

Ошибки

Для публичных и внутренних endpoint-ов сохраняется единый формат ошибки. Он совместим с ошибками монолита: **VALIDATION**, **UNAUTHORIZED**, **FORBIDDEN**, **NOT_FOUND**, **CONFLICT** и **INTERNAL**.

```
{
  "code": "FORBIDDEN",
  "message": "Employer is not assigned to this vacancy",
  "details": {
    "vacancyId": "71d50a83-475a-44cf-a9ab-048b3e9b80a4"
  }
}
```

HTTP	Код	Когда возникает
400	VALIDATION	Некорректный body, query-параметр или неподдерживаемое поле поиска/сортировки
401	UNAUTHORIZED	Нет JWT или X-Service-Token для внутренней ручки
403	FORBIDDEN	Недостаточно прав или пользователь не владеет ресурсом
404	NOT_FOUND	Пользователь, компания, вакансия, резюме или отклик не найдены
409	CONFLICT	Повторный отклик, повторное избранное, конфликт primary assignment
500	INTERNAL	Необработанная серверная ошибка

План миграции от монолита

- Зафиксировать текущие публичные API-контракты и покрыть ключевые сценарии интеграционными тестами.
- Вынести api-gateway и настроить маршрутизацию к текущему монолиту как к временной backend-цели.
- Выделить auth-service и user-service: перенести регистрацию, логин, пользователей и роли.
- Выделить company-service: перенести компании и employer profiles.
- Выделить vacancy-service: перенести вакансии, назначения HR и справочники вакансий.
- Выделить search-service: построить поисковую проекцию вакансий и переключить GET /vacancies на него через gateway.
- Выделить resume-service и перенести резюме соискателей.
- Выделить application-service: перенести отклики, статусы и приглашения на собеседование.
- Выделить interaction-service: перенести избранное и историю просмотров.
- Добавить notification-service и подключить его к Kafka events.
- После стабилизации удалить оставшиеся доменные модули из монолита и оставить только микросервисные контракты.

Результат

В результате проектирования получена целевая архитектура из десяти микросервисов, соответствующая принципу database-per-service. Сервис вакансий и сервис поиска разделены по CQRS: vacancy-service остается источником правды, а search-service строит асинхронную поисковую read-модель. Kafka используется для обновления поискового индекса, уведомлений и будущей аналитики. Для синхронных проверок подготовлен отдельный OpenAPI-контракт внутренних endpoint-ов.

Вывод

Предложенное разбиение сохраняет баланс между микросервисной декомпозицией и практичностью. В архитектуре отсутствует отдельный dictionary-service, так как справочники не являются самостоятельным бизнес-доменом. При этом search-service выделен отдельно, потому что поиск вакансий имеет собственную read-модель, может масштабироваться

независимо и в будущем может быть переведен с PostgreSQL на OpenSearch или Elasticsearch без изменения доменной модели vacancy-service.